

SUPSI

Branch and Bound as PTAS

Studente/i

Tiziano Zamaroni

Sergio Aquilini

Relatore

Monaldo Mastrolilli

Correlatore

-

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

**M-P6050P - Progetto di
Semestre**

Anno

2025

Data

11 dicembre 2025

Indice

1	Introduzione	1
2	Motivazione e Contesto	3
2.1	Motivazione	3
2.2	Contesto	3
3	Algoritmi	5
3.1	Approssimazione soluzione	5
3.2	Branch-and-Bound	5
3.3	Greedy	5
3.4	Profiling	5
4	Requisiti	7
4.1	Requisiti funzionali	7
4.2	Requisiti non funzionali	8
5	Tecnologie utilizzate	9
5.1	Implementazione algoritmi	9
5.2	Analisi dei dati	9
6	Implementazione	11
6.1	Architettura e struttura del codice	11
6.2	Branch-and-Bound	11
6.2.1	Overview of the Framework	11
6.2.2	Pseudocode	14
6.2.3	Lower-Bound Strategies	15
6.2.3.1	Linear relaxation (lin_relax).	15
6.2.3.2	Binary search on the makespan (bin_search).	15
6.2.4	Branching Rules	16
6.2.4.1	Maximum minimum processing time (max_min_proc).	16
6.2.4.2	Maximum average processing time (max_avg_proc).	16

6.3 Greedy	17
6.4 Profiling	17
6.5 Generazione istanze	17
6.6 Analisi dei dati	17
7 Analisi dei risultati	19
8 Metodologia di lavoro	21
9 Conclusione	23
9.1 Risultati ottenuti	23
9.2 Possibili miglioramenti futuri	23

Elenco delle figure

Elenco delle tabelle

1 Introduzione

- Problemi NP-Hard
- Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes
- Unrelated parallel machine scheduling problem
- Uniform/Identical parallel machine scheduling problem (identical caso particolare di uniform)
- Profiling come ulteriore ottimizzazione di identical machine scheduling
- Dimostrazione pratica della validità dell'approccio con profiling

2 Motivazione e Contesto

2.1 Motivazione

2.2 Contesto

3 Algoritmi

3.1 Approssimazione soluzione

Differenze del valore di epsilon tra i vari problemi

3.2 Branch-and-Bound

3.3 Greedy

3.4 Profiling

4 Requisiti

Il lavoro di semestre si inserisce nel contesto del paper *Branch-and-Bound Algorithms as Polynomial-time Approximation Scheme* [1], scritto dal nostro relatore Prof. Monaldo Mastroilli insieme a István Encz Koppányi ed Eleonora Vercesi. Gli autori forniscono un'implementazione Python del Branch-and-Bound per il caso delle *unrelated machines*, comprensiva di euristiche e gestione dei bound. Il nostro compito consiste nel comprendere a fondo tale implementazione, adattarla al caso delle *identical machines* e integrare la tecnica di *profiling*, applicabile poiché le identical machines rappresentano un caso particolare di uniform machines. L'obiettivo finale è verificare sperimentalmente quanto presentato nel paper, confrontando la versione base dell'algoritmo con quella dotata di profiling.

4.1 Requisiti funzionali

Il progetto richiede i seguenti requisiti funzionali:

RF1 – Solver per identical machines

Realizzare una variante dell'algoritmo Branch-and-Bound in grado di risolvere il problema di scheduling su identical machines, adattando la struttura e le funzioni dell'implementazione esistente per le unrelated machines.

RF2 – Integrazione del profiling

Implementare il meccanismo di profiling descritto nel paper, così da ottenere una seconda variante dell'algoritmo direttamente confrontabile con la versione base.

RF3 – Generazione ed esecuzione degli esperimenti

Eseguire entrambe le varianti dell'algoritmo e registrare i risultati, raccogliendo una quantità consona di dati.

RF4 – Raccolta dei dati

Memorizzare in formato strutturato informazioni quali nodi esplorati, tempo di esecuzione, delta rispetto alla soluzione ottima dell'istanza considerata e altre metriche rilevanti.

RF5 – Validazione sperimentale

Analizzare i dati raccolti, con l'aiuto di strumenti statistici e di visualizzazione, per verificare l'efficacia del profiling rispetto alla versione base dell'algoritmo.

4.2 Requisiti non funzionali

Non erano previsti requisiti non funzionali formali, ma il progetto implicava alcuni vincoli pratici:

Efficienza computazionale

L'implementazione e la scelta delle istanze dovevano permettere di generare e analizzare un numero significativo di dati entro i limiti temporali del lavoro di semestre.

Ripetibilità

La generazione delle istanze e l'esecuzione degli algoritmi dovevano essere riproducibili tramite seed controllati.

Mantenibilità e pulizia del codice

Anche trattandosi di un esperimento non destinato a evoluzione futura, il codice doveva essere scritto in modo chiaro e integrarsi coerentemente con la struttura esistente.

5 Tecnologie utilizzate

5.1 Implementazione algoritmi

L'implementazione degli algoritmi è stata sviluppata in *Python* a partire dal codice fornito dagli autori del paper [1], originariamente progettato per il caso delle *unrelated machines*. In tale versione, il lower bound dei nodi del Branch-and-Bound veniva calcolato tramite il solver *SCIP* [2], utilizzato attraverso la libreria *PySCIPOpt*. Questa scelta, appropriata per il problema generale, consente agli autori di ottenere bound frazionari mediante *binary search* e *linear relaxation*.

Nel nostro progetto, tuttavia, tale approccio si è rivelato eccessivamente oneroso e non strettamente necessario. Nel caso delle *identical machines*, infatti, la struttura del problema è significativamente più semplice e permette l'impiego di metodi più leggeri per il calcolo del lower bound. Inoltre, l'utilizzo di *SCIP* [2] presentava un ulteriore limite pratico: non è stato possibile forzare il solver a utilizzare sistematicamente il metodo *primal simplex*, condizione necessaria per garantire che il numero di variabili frazionarie nella soluzione LP rimanesse inferiore o uguale al numero di macchine. L'assenza di tale garanzia comprometteva l'affidabilità della fase di arrotondamento e introduceva un overhead computazionale non trascurabile.

Per queste ragioni, abbiamo sostituito integralmente l'utilizzo di *SCIP* [2] con un metodo di calcolo del lower bound basato su un criterio *greedy*, pienamente sufficiente per il caso delle *identical machines* e decisamente più efficiente dal punto di vista computazionale. Questa scelta ha permesso di ridurre drasticamente il tempo di elaborazione dei nodi del Branch-and-Bound, mantenendo comunque una qualità del bound adeguata agli scopi del progetto.

5.2 Analisi dei dati

L'analisi dei dati sperimentali è stata effettuata in *Python* utilizzando le principali librerie per la data science. *numpy* è stato impiegato per l'elaborazione numerica di base, *pandas* per l'organizzazione e l'aggregazione dei risultati raccolti durante gli esperimenti, e *matplotlib* per la generazione delle visualizzazioni utilizzate nel confronto tra la versione base

dell'algoritmo e quella dotata di profiling. I dati sono stati esportati in formato CSV e analizzati tramite notebook dedicati, permettendo di sintetizzare l'andamento delle performance e supportare quantitativamente la discussione dei risultati.

6 Implementazione

6.1 Architettura e struttura del codice

6.2 Branch-and-Bound

The implementation employed in this project is based on a branch-and-bound (B&B) framework originally developed for the *unrelated machines scheduling* problem. The same structure is later reused for the identical machines case, where only the lower-bound routines and certain optimisation components (such as profiling) are adapted, while the core branching, bounding, rounding, pruning, and node-selection logic remains unchanged.

6.2.1 Overview of the Framework

Each node of the B&B tree represents a partial assignment of jobs to machines and is defined by:

- a set of fixed assignments $\text{fixed} \subseteq \{0, \dots, n-1\} \times \{0, \dots, m-1\}$,
- an overhead vector $h \in \mathbb{R}^m$ containing the machine loads induced by these assignments,
- a fractional solution X^{frac} obtained from a relaxation of the problem,
- a node lower bound LB and a node upper bound UB,
- the depth of the node in the search tree.

The global state maintains a best-known integer solution (global upper bound LUB) and a global lower bound LLB derived from active nodes. Four main components determine the behaviour of the algorithm:

Bounding

A lower bound is computed for each node by solving either a linear relaxation or a feasibility-based binary search. Nodes whose lower bound is not better than the current LUB are immediately pruned.

Branching

If the fractional relaxation contains non-integer job assignments, one fractional job is selected according to a branching rule. This job is then fixed on each possible machine, producing up to m child nodes.

Rounding

A feasible integer solution is generated from the fractional relaxation using a deterministic rounding strategy. This solution yields an upper bound (UB) for the node and may improve the global best solution.

Node selection

Nodes are stored in a priority queue, ordered according to a configurable strategy (best lower bound first, depth-first, breadth-first). This choice influences the traversal order but not correctness.

The algorithm proceeds by repeatedly extracting the next node from the queue, updating global bounds, checking for termination, branching on fractional jobs, and processing child nodes until either the relative optimality gap satisfies

$$\frac{\text{LUB}}{\text{LLB}} < 1 + \varepsilon$$

, or the search space has been fully explored. An hard limit on the number of nodes explored may also be imposed to prevent excessive runtimes.

6.2.2 Pseudocode

Algorithm 1 Branch-and-bound for unrelated machines scheduling (inherited framework)

Require: processing times $p_{j,i}$, node selection strategy, lower-bound strategy, branching rule, rounding rule, tolerance ε

```

1: Initialize global bounds:  $LUB \leftarrow +\infty, LLB \leftarrow -\infty$ 
2: Set root depth  $d = 0$ , overhead  $h = (0, \dots, 0)$ , and fixed assignments  $fixed = \emptyset$ 
3: Compute root lower bound:  $(X^{frac}, LB) \leftarrow \text{LowerBound}(p, h, fixed)$ 
4: if  $X^{frac}$  is integer then
5:    $LUB \leftarrow LB$ 
6:   return optimal solution found at the root
7: end if
8: Compute initial feasible solution:  $(X^{int}, UB) \leftarrow \text{Rounding}(X^{frac}, fixed)$ 
9:  $LUB \leftarrow UB$ 
10: Create root node  $N_0$  and insert it into the priority queue  $Q$ 
11: while  $Q \neq \emptyset$  do
12:   Extract next node  $N = (X^{frac}, LB, d, fixed, h, X^{int}, UB)$ 
13:   Update global lower bound:

```

$$LLB \leftarrow \min(LUB, \min_{N' \in Q} LB(N'))$$

```

14:   if  $LUB/LLB < 1 + \varepsilon$  then
15:     return current best solution
16:   end if
17:   Select branching job  $j$ 
18:   for each machine  $q \in \{0, \dots, m-1\}$  do
19:     Create updated overhead  $h'$  and fixed set  $fixed'$ 
20:     if all jobs are fixed then
21:       Compute  $UB' = \max_i h'_i$ 
22:       if  $UB' < LUB$  then update global best solution
23:     end if
24:     continue
25:   end if
26:   Compute child lower bound:  $(X^{frac'}, LB') \leftarrow \text{LowerBound}(p, h', fixed')$ 
27:   Apply fixed assignments to  $X^{frac'}$ 
28:   if  $LB' \geq LUB$  then prune node
29:   end if
30:   if  $X^{frac'}$  is integer then
31:     if  $LB' < LUB$  then update global best solution
32:   end if
33:   continue
34:   end if

```

Branch and Bound as PTAS

```

35:   Obtain feasible integer solution:  $(X^{int'}, UB') \leftarrow \text{Rounding}(X^{frac'}, fixed')$ 
36:   if  $UB' < LUB$  then update global best solution
37:   end if
38:   Insert new node into  $Q$ 

```

6.2.3 Lower-Bound Strategies

The inherited implementation provides two alternative lower-bound computations for each node of the branch-and-bound tree. Both methods operate on the subproblem defined by:

- the set of fixed assignments fixed ,
- the overhead vector $h \in \mathbb{R}^m$, where h_i is the load already assigned to machine i ,
- the remaining unfixed jobs.

In both cases, the result is a fractional assignment X^{frac} and a scalar lower bound LB.

6.2.3.1 Linear relaxation (`lin_relax`).

In the linear-relaxation strategy, a linear program is solved in which each unfixed job may be fractionally assigned to machines. Let $x_{j,i} \geq 0$ denote the fraction of job j assigned to machine i , and let $C_{\max}$ denote the makespan. The model can be expressed as:

$$\begin{aligned}
 & \min C_{\max} \\
 & \text{s.t.} \quad \sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed,} \\
 & \quad \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \leq C_{\max} - h_i && \forall i = 0, \dots, m-1, \\
 & \quad x_{j,i} \geq 0 && \forall j, i.
 \end{aligned}$$

The fixed jobs do not appear as decision variables; their contribution is encoded in the overhead h . The optimal value $C_{\max}^*$ is used as the node lower bound LB, and the corresponding solution defines X^{frac} . Due to the structure of the formulation, at most m jobs are fractional in the optimal solution.

6.2.3.2 Binary search on the makespan (`bin_search`).

The second strategy performs a binary search on the makespan value. An interval $(\ell, r]$ is maintained such that ℓ is infeasible and r is feasible. For a candidate makespan C , a feasibility LP is solved with variables $x_{j,i}$ for unfixed jobs, subject to:

$$\begin{aligned}
 & \sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed,} \\
 & \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \leq C - h_i && \forall i = 0, \dots, m-1, \\
 & x_{j,i} = 0 && \text{if } p_{j,i} > C - h_i, \\
 & x_{j,i} \geq 0 && \text{otherwise.}
 \end{aligned}$$

If the LP is feasible, the value C becomes the new right endpoint; otherwise it becomes the new left endpoint. The process continues until $(\ell, r]$ cannot be further refined, and the final feasible value r is taken as the node lower bound. The corresponding LP solution defines the fractional assignment X^{frac} .

6.2.4 Branching Rules

When the relaxation at a node produces a non-integer solution, a set of *fractional jobs* is obtained, namely those jobs j for which at least one variable $x_{j,i}$ is non-integer. The implementation assumes that the number of such jobs is at most the number of machines m , which is guaranteed by the structure of the relaxations. A branching rule selects one job j from this set, and the node is branched by fixing job j on each machine $q = 0, \dots, m-1$, generating up to m children.

Two selection criteria are provided.

6.2.4.1 Maximum minimum processing time (`max_min_proc`).

For each fractional job j , the quantity

$$\min_{i=0, \dots, m-1} p_{j,i}$$

is considered. The branching job is the one that maximises this minimum processing time. This rule gives priority to jobs that are relatively expensive even on their best machine, so that branching decisions focus on jobs with high potential impact on the makespan.

6.2.4.2 Maximum average processing time (`max_avg_proc`).

For each fractional job j , the average processing time

$$\frac{1}{m} \sum_{i=0}^{m-1} p_{j,i}$$

is computed. The branching job is the one with maximum average processing time, i.e., the job that is globally heaviest across all machines. This tends to branch first on jobs that dominate the total processing workload.

6.3 Greedy

6.4 Profiling

6.5 Generazione istanze

6.6 Analisi dei dati

7 Analisi dei risultati

- Overview istanze e dati raccolti
- Istanze relativamente piccole ma con diversi seed
- Difficoltà variabile a dipendenza del seed, a parità di dimensione
- Statistiche no-profiling vs. prune per tipo di istanza
- Evidenziare che profiling rimane percorribile quando la versione base non arriva a soluzione

8 Metodologia di lavoro

- Suddivisione dei compiti
- Pianificazione e Task
- Pair programming / brainstorming
- Weekly
- Raccolta dei dati

9 Conclusione

9.1 Risultati ottenuti

9.2 Possibili miglioramenti futuri

Elenco di possibili sviluppi futuri:

- abc
- def

Bibliografia

- [1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.
- [2] Scip optimization suite. <https://scipopt.org>.